Aarhus University

Project number - 2026F25

Asymmetrical Perspectives

Students:
Thea Teresa Lindgaard (202204350)
Niels Jakob Østerberg (202105879)
Kasper Stecher (202100451)
Kasper Bentsen (202103249)

Counsellor:
Lukas Esterle

Aarhus, May 29th 2026

Contents

Author: Thea Teresa Lindgaard

1. Introduction

The game is a cooperative multiplayer puzzle game, centered around communication between the two players. The player roles, cat and human, perceive the environment in the game differently. Each level is built around the idea, that the players must communicate what they see to each other in order to complete the level.

The systems described in this document fulfills functional requirement **E2-US4: Asymmetrical Perspective**.

1.1. Defining Asymmetry

Visual asymmetry can mean many things. To set the scope of the asymmetry, different kinds of asymmetry were defined in more detail, along with the puzzle-building opportunities they each provide:

1. **Light Asymmetry**

   Each player's scene is uniquely lit. This means each player may experience different light levels and/or colors.

   This can neatly fit into the game concept, as cats are known to have excellent night vision. It allows for building puzzles, where the cat is the only player with enough light to properly see their surroundings.

2. **Geometry and Material Asymmetry**

   Each player perceive an object with different geometry meshes. For example, one player sees a sphere, while the other sees a cube. In unity, every 3D object has a material, that determines its texture and color. This can also be made to look different for each player.

   An item such as a key clearly communicates it's purpose and importance to a human, but might not seem important to a cat, who doesn't understand its use.

   With geometry and material asymmetry, this difference in understanding and perception of the world can be expressed in game-play by letting the key literally look like something ordinary to the cat, such as a cat toy.

1. **Visibility Asymmetry**

   One player sees an object that is invisible to the other.

   This opens up puzzle-building opportunities that are similar to the above. One player has access to information that the other player can only get through verbal communication. Players must describe what they can see to each other in order to solve puzzles.

2. **Collider Asymmetry**

   One player does not collide with an object, but the other does.

   This can be utilized to let only one player pass through things, thus creating separated zones designed for each role.

While light, geometry, material and visibility asymmetry can be used to represent the difference in a human's and cat's perception of the world, collision asymmetry lean more into the supernatural, and might cause strange visual artifacts, as players appear to pass through objects. For this reason, collisions were not made asymmetrical in the final draft of the game.

In general, physics should remain consistent for both players. That means that objects are in the same position, with the same rotation and scale, and whose physics are simulated in the same way (fx. same gravity). The desired outcome of asymmetry is exclusively perceptual.

2. Design

The following sections describe the design constraints and design choices made when implementing the asymmetry system.

2.1. How Players Experience Asymmetry

After establishing a multiplayer connection, players enter a shared lobby where they can freely select a level to play. This is also the only place, where players are able to swap roles.

When the players enter a level, they each experience the environment according to their assigned role. The level's perspective remains fixed throughout the level. As a result, asymmetry does not change dynamically while a level is active.

To change perspective, players must return to the lobby, swap roles, and re-enter the level, at which point the scene is reloaded, and the perspectives will be swapped.

2.2. Design Goals and Constraints

The asymmetry system is designed to support a cooperative experience in which two players perceive the world differently. A key design constraint, is that both players must share the same logical world state. This ensures that interactions like movement, physics and object placement remain consistent. In other words, the asymmetry system does not affect gameplay-relevant state, in only acts as a perceptual layer.

This ensures that interactions remain consistent across players, allowing both to respond to physics-based events using the same underlying rules.

The previously established gameplay mechanics lead to the following architectural requirements:

- Perspective depends on the role of the local player
- Levels must initialize asymmetry each time the level loads according to the player's role
- Perspective does not change while a level is active
- Asymmetry exists on a per-object basis, with individual objects optionally defining unique appearances per role
- Every object can optionally define its own asymmetry behavior
- A central controller must be able to dynamically discover all asymmetrical objects in the scene and apply the appropriate perspective

In addition, the asymmetrical system is designed be extensible with new types of asymmetry. This allows the introduction of new mechanics later on, without requiring changes to existing code.

3. Implementation

The following sections describe how the asymmetry system was implemented.

3.1. Strategy

A component-based strategy is greatly preferred over alternatives such as building two parallel versions of each level, because most level content is shared. A component-based strategy reduces duplication, lowers the risk of human error during level-building, and eases the workload of level-building significantly.

It is also preferred over shader-based strategies or layering tricks. Shader- or layer-based asymmetry distributes the asymmetry responsibility across cameras, render pipelines, layers, scene configurations etc. Additionally, it introduces a significant burden when building levels, as level designers would have to manually configure cameras, manage layers, ensure render consistency and more.

The component-based system allows level designers to attach an asymmetry component to a GameObject, and configure values for the cat and human perspectives through the Unity Editor. The component-based architecture is less complex, and has better usability compared to the alternatives.

3.2. Architecture

The system is implemented using a component-based architecture. This enables modular, per-object control over asymmetrical objects. As only an object's transform component is synchronized across players, it is safe to modify any other component at runtime to achieve asymmetry.

This ensures that the differences between players remain purely visual. In addition, it avoids duplication, because players share the same scene, with the same GameObjects, as opposed to near-identical scenes with pre-baked asymmetry.

For controlling component values runtime, a two-layer abstraction was implemented consisting of one non-generic, and a derived generic base class, representing an asymmetrical property of a scene object. These are responsible for finding and modifying the component according to a given player role.

To manage all asymmetrical objects, a main controller (PerspectiveManager) was implemented. This class dynamically locates all asymmetric objects, and ensures that they apply a perspective at the right time. This way, individual objects aren't responsible for their own state transitions.

Initialization of perspectives is triggered externally by the LevelManager class. See **Figure 1** for a diagram over the classes involved.

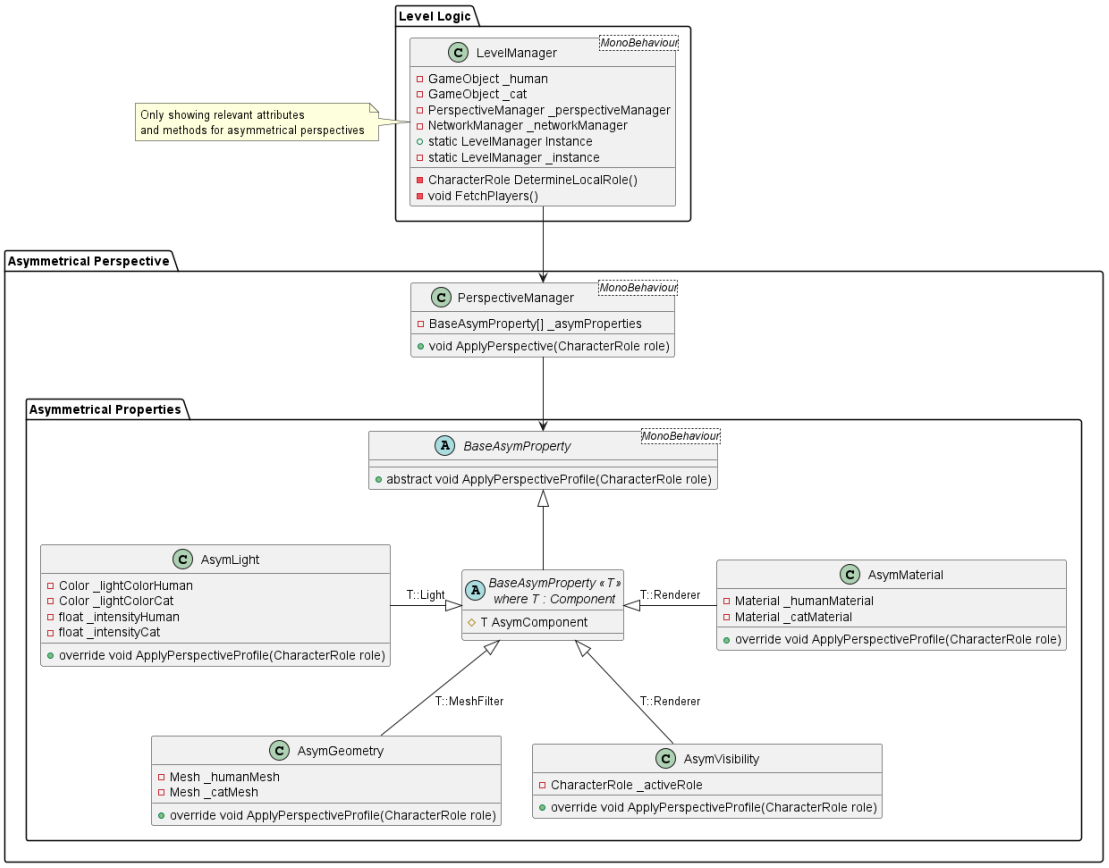

Figure 1: Class diagram showing the classes involved with perspective handling

This design choice offers several benefits:

1. The non-generic base class functions as a common abstraction of all asymmetrical components. It allows the central controller (PerspectiveManager) to handle all asymmetrical properties uniformly, with no type-specific handling required.

2. The generic base class provides common logic for dynamically fetching the component being managed from the GameObject, reducing duplication across concrete implementations.

3. Although the non-generic base class does not define any shared behavior, it was preferred over an interface, because it integrates better Unity's serialization and scene management systems. Specifically, it allows the PerspectiveManager to dynamically query the scene for all objects assignable to the non-generic base class.

3.3. Concrete Implementations

The generic base class implements the following common behavior for fetching the component:

```
private void Awake()
{
   AsymComponent = GetComponent<T>();
   if (AsymComponent == null)
   {
       throw new NullReferenceException($"'{gameObject.name}' is trying to access a
'{typeof(T)}' component, but GameObject has no such component attached");
   }
}
```

Each AsymComponent implements the method ApplyPerspective, that alters the component according to the values set in the inspector. Below is an example of how the AsymGeometry applies the perspective:

```
public override void ApplyPerspectiveProfile(CharacterRole role)
{
    AsymComponent.mesh = role switch
    {
        CharacterRole.Human => _humanMesh,
        CharacterRole.Cat => _catMesh,
        _ => throw new ArgumentException("Unexpected CharacterRole")
    };
}
```

The PerspectiveManager utilizes Unity's FindObjectsByType method to dynamically locate all AsymProperty objects in the scene. This way any number of AsymProperties can be added or removed, without the need to manually maintain references to these objects.

```
public class PerspectiveManager : MonoBehaviour
{
    private BaseAsymProperty[] _asymProperties;

    private void Awake()
    {
        _asymProperties =
FindObjectsByType<BaseAsymProperty>(FindObjectsSortMode.None);
    }

    public void ApplyPerspective(CharacterRole role)
    {
        // apply perspective settings to every asymmetrical object in the scene
        foreach (BaseAsymProperty asymProperty in _asymProperties)
        {
            asymProperty.ApplyPerspectiveProfile(role);
        }
    }
}
```

The perspective manager serves as a central controller, and ensures all objects update their state at the same time. This was chosen over a distributed strategy, where individual objects manage their own state for the following reasons:

- Individual objects don't need to know about which player is local, and this player's role, or define how to find it
- It gives finer control over when perspectives change, making it independent of defined level flow rules. This way, the system is still usable, should the level flow change
- Ensures consistency across all objects, and grants a single source of truth

3.4. Initialization Order

The perspective is automatically applied as the scene loads using Unity's lifecycle methods. Awake, which runs first, is used to fetch all required references to other classes or components. As the execution order of Awake is not guaranteed, this step ensures all references are resolved before further initialization.

In Start, which runs before the first frame, the perspective is configured for all asymmetrical objects in the scene. See **Figure 2** for a sequence diagram displaying the described flow.

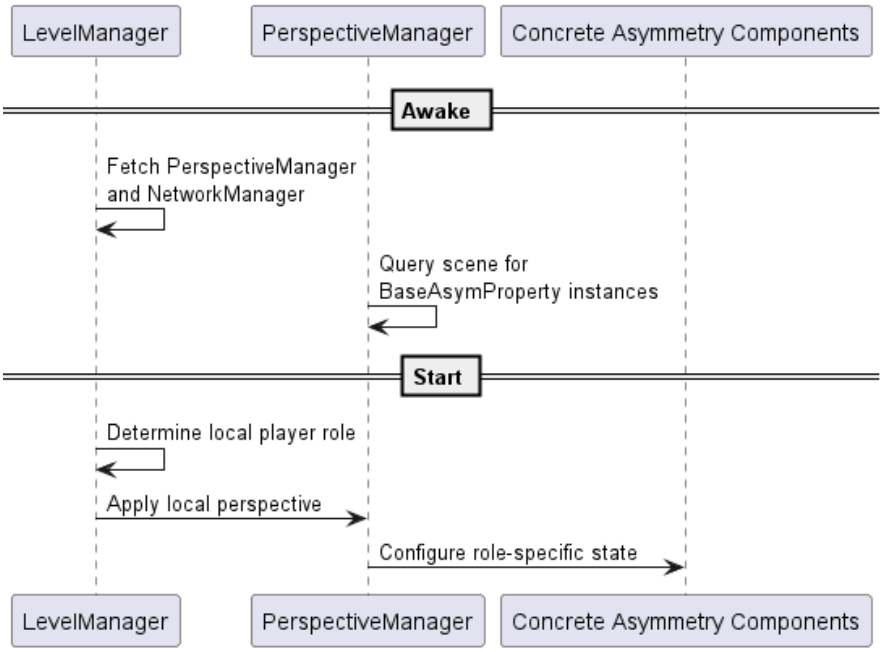

Figure 2: Sequence diagram showing the initialization order for perspective handling

3.5. Limitations and Trade-offs

This design prioritizes flexibility and modularity, at the cost of initialization overhead. Since components are discovered at runtime, a slight overhead is introduced at the start of each level. However, since levels are relatively small, and since the cost of initialization only affects initial load times, it was considered an acceptable trade-off.

Additionally, the system assumes that perspectives remain static throughout a level. Currently, this is appropriate, because this is the intended gameplay experience. If this should change, the system might need to be re-evaluated.

The cost of applying, or re-applying, perspectives could cause the game to lag, or cause long load times, especially if levels grow considerably in size and number of asymmetric components. In that case, the system should be reworked to distribute the workload, for example by applying perspective to objects within the camera, while a level is active.

3.6. Extensibility

Any new types of asymmetry can be added with no impact to the current system, by implementing the generic base class. The new asymmetry type will be automatically discovered if it exists in the scene, and since it defines its own asymmetry behavior, it can be treated like any of the existing components.

This helps support long term scalability and development of the game, and allows for easy introduction of new mechanics related to asymmetry.

3.7. Concrete Asymmetric Properties

Unity's light component has several configurable parameters. In this implementation, color and intensity have been made asymmetrical. Making the color of light asymmetrical represents the difference in color perception between humans and cats, while light intensity represents the difference in night vision. Other parameters remain shared to reduce complexity and maintain inspector clarity.

Similarly, asymmetrical material and geometry changes that attached material or the mesh of the mesh filter component at runtime.

The AsymVisibility component simply disables the renderer for an object entirely to make it invisible.

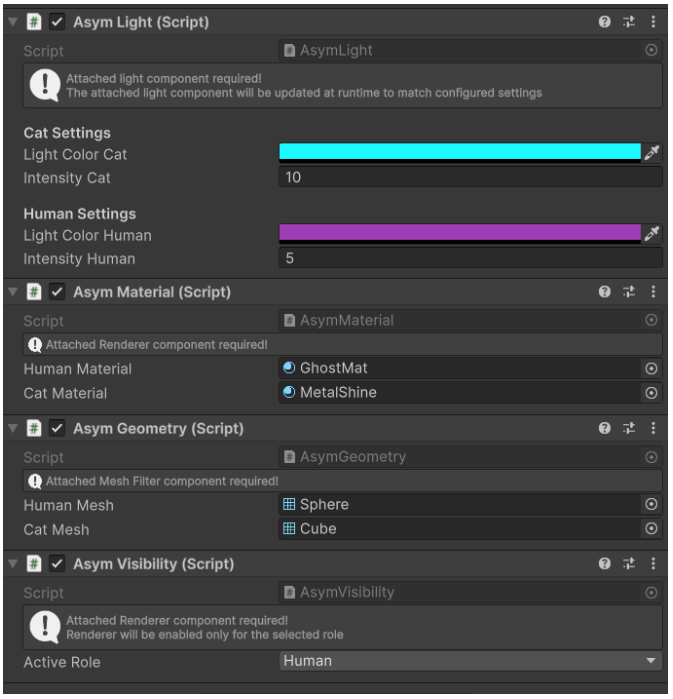

Figure 3: Inspector view of all AsymProperty components, showing separate configuration parameters for human and cat perspectives.

4. Verification

The following sections describe how the system was tested, and the results.

4.1. Testing Method

The asymmetry system was manually tested through runtime inspection in the Unity Editor. This was deemed approprite because:

- The system is heavily reliant on engine-integrated features such as scene management, lifecycle events and rendering, which can’t be easily mocked
- Correctness is not easily reduced to structured automated tests and deterministic results, as they are visual as much as logical.
- Perspectives had to be tested in a multiplayer setting, to ensure that no unintended synchronization was breaking the asymmetry
- Unit tests only verifies internal state, not the rendered outcome

To verify correctness, a test scene with all asymmetry types represented was manually inspected from both player perspectives.

4.2. Test Setup

A test scene was created to validate the asymmetry system. The scene contained only white objects, so that the light can properly show in color and intensity. The scene was made completely dark for the same reason.

The objects within the scene are primitive shapes, so the asymmetry shows clearly in both geometric structures and material. The test objects are:

- A sphere with an AsymVisibility component attached, configured to only be visible to the human player
- A sphere with an AsymMaterial component, configured with a reflective white material for the human perspective, and a sheer white material for the cat perspective
- An object with an AsymGeometry, configured with the sphere mesh for the human, and the cube mesh for the cat
- Three light sources with different light types, configured with cold colors and low intensity for the human, and warm colors and high intensity for the cat

The scene was integrated into the existing level loading system, and loaded through the lobby, to confirm that the level configuration can correctly integrate with the other systems within the game.

4.3. Results

In **Figure 4** is a screenshot from the scene, viewed from the human perspective. The scene is dimly lit with predominantly pink and purple hues. The sphere on the left is visible. The sphere in the center appears shiny and smooth. The object on the right is a square.

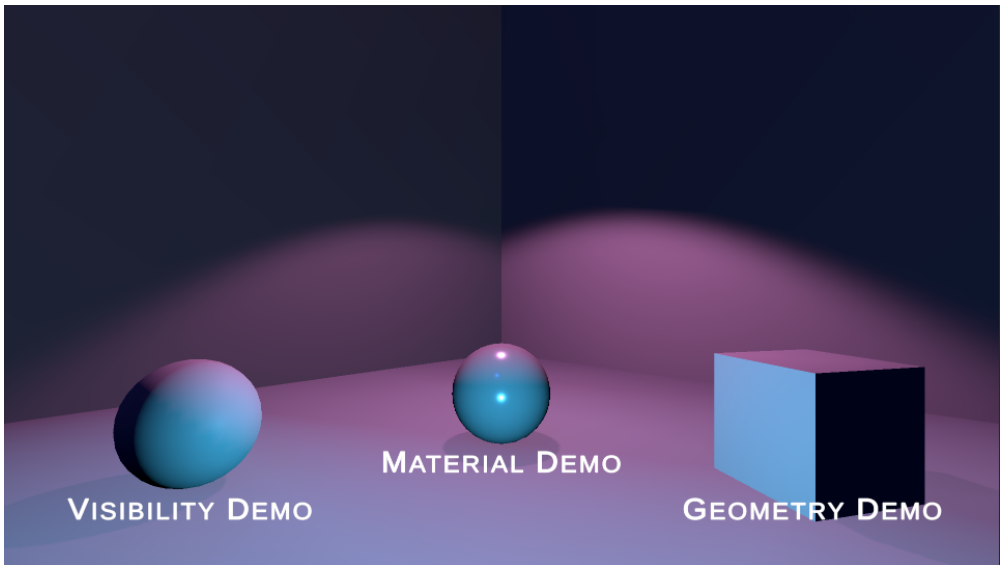

Figure 4: A screenshot from the testing scene showing the human player's view

In **Figure 5** is a screenshot from the same scene, from the cat perspective. The scene bright, with green and yellow tones. The object on the left is completely invisible. The object in the center is transparent, and no longer shiny. The object on the right is a sphere, not a square.

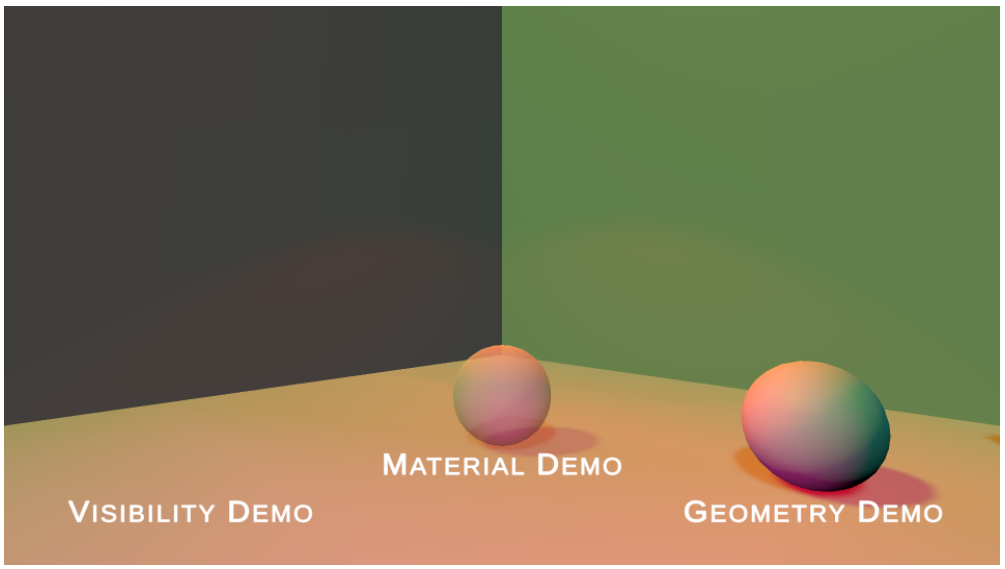

Figure 5: A screenshot from the testing scene showing the cat player's view

Based on these observations, it was concluded, that the system behaved as intended.

5. Conclusion

The resulting asymmetry system fits the requirements well. It required little configuration during level building, and integrated easily with other game systems through a single control point.

The system can be easily extended with new asymmetry types, by implementing the generic AsymProperty base class. All asymmetry components in the scene are automatically discovered, eliminating the need for manually maintained references, and thus reducing the risk of human error.

For this game prototype, the system is well designed and sufficiently optimized. However, the system does not scale well, if new player roles are added. New player roles would require that all asymmetric components get updated. This is a trade-off, making the system more easily extendable with new asymmetry types, costs extensibility for new player roles.

Additionally, the automatic discovery of asymmetry objects currently find all relevant objects at once, using one big search at the beginning of the level. For the game prototype, this is sufficient. However, should levels increase dramatically in number of GameObjects, it is possible that level load times becomes too long. In this case, alternatives such as chunk based loading or baked data should be considered.